Lec16

Synchronous Javascript

JavaScript executes code **line by line**.

Example:

```
console.log("Start");
console.log("Middle");
console.log("End");
```

Output:

```
Start
Middle
End
```

The next statement waits until the previous statement finishes.

This is called **Synchronous Programming**.

Problems with Synchronous Programming

Imagine downloading a file that takes 10 seconds.

```
downloadFile();
showData();
```

If JavaScript waits for download completion:

Download starts
(wait 10 seconds)
Download completed
Show Data

The application becomes slow and unresponsive.

To solve this problem JavaScript uses:

- Callbacks
- Promises
- Async/Await

These are collectively called **Asynchronous Programming**.

Asynchronous Javascript

Asynchronous programming allows JavaScript to start a task and continue executing other code without waiting for that task to finish.

Example:

```
console.log("Start");

setTimeout(() => {
   console.log("Downloaded");
}, 3000);

console.log("End");
```

Output:

```
Start
End
Downloaded
```

JavaScript does not wait for the timer.

Why Asynchronous Programming?

Used for:

- API Calls
- Database Queries
- File Uploads
- Downloads
- Timers
- User Events
- Network Requests

Callbacks

A callback is a function passed as an argument to another function and executed later.

Example

```
function greet(name, callback) {
   console.log("Hello " + name);
   callback();
}
function goodbye() {
   console.log("Goodbye");
}
greet("Daksh", goodbye);
```

Why use Callbacks?

Suppose a file takes time to download.

```
downloadFile(function() {
   showData();
});
```

When download finishes, callback executes.

Callback with setTimeout

```
console.log("Download Started");

setTimeout(() => {
   console.log("Download Completed");
}, 3000);
```

Output:

```
Download Started
(wait)
Download Completed
```

Callback Hell

A major problem of callbacks.

Example:

```
loginUser(function(user){
   getProfile(user,function(profile){
      getPosts(profile,function(posts){
         getComments(posts,function(comments){
            console.log(comments);
         });
      });
   });
});
```

This is called:

Pyramid of Doom

or

Callback Hell

Problems:

- Hard to read
- Difficult debugging
- Difficult maintenance
- Deep nesting

Promises were introduced to solve this.

Promises

A Promise represents a future value.

Think:

Promise = "I will give you the result later."

A Promise can be:

Pending

Task still running.

Fulfilled

Task completed successfully.

Rejected

Task failed.

Creating a Promise

```
const promise = new Promise((resolve, reject) => {

let success = true;

if(success){
resolve("Task Completed");
}
else{
reject("Task Failed");
}

});
```

Consuming a promise

```
promise
    .then(result => {
        console.log(result);
    })
    .catch(error => {
        console.log(error);
    });
```

Example

```
const downloadFile = new Promise((resolve) => {

setTimeout(() => {

resolve("File Downloaded");

}, 3000);

});
```

then()

Runs after success.

```
downloadFile.then(result => {
   console.log(result);
});
```

catch()

Runs after failure.

```
downloadFile.catch(error => {
   console.log(error);
});
```

finally()

Runs always.

```
downloadFile
   .finally(() => {
      console.log("Finished");
   });
```

Promise Chaining

```
loginUser()
.then(user => getProfile(user))
.then(profile => getPosts(profile))
.then(posts => getComments(posts))
.then(comments => console.log(comments))
.catch(error => console.log(error));
```

Promise Methods

Promise.all()

Runs multiple promises together.

```
Promise.all([
  fetchUsers(),
  fetchPosts(),
  fetchComments()
])
```

Returns when all succeed.

Promise.race()

Returns first completed promise.

```
Promise.race([
   promise1,
   promise2
])
```

Promise.any()

Returns first successful promise.

```
Promise.any([
   promise1,
   promise2
])
```

Promise.allSettled()

Returns all results regardless of success or failure.

```
Promise.allSettled([
   promise1,
   promise2
])
```

Async/Await

Makes asynchronous code look synchronous.

Most commonly used in modern applications.

async Keyword

An async function always returns a Promise.

```
async function greet() {
   return "Hello";
}
```

Actually returns:

Promise.resolve("Hello")

await Keyword

Used inside async functions.

Pauses execution until Promise resolves.

Example

```
async function getData() {

   const data = await fetchData();

   console.log(data);

}
```

Looks synchronous but is asynchronous internally.

Example with Promise

```
function fetchData() {

return new Promise(resolve => {

setTimeout(() => {

resolve("User Data");

}, 2000);

});

}
```

Using Async/Await

```
async function getData() {

const result = await fetchData();

console.log(result);

}

getData();
```

Error Handling

Promise Style

```
fetchData()
.then(data => console.log(data))
.catch(error => console.log(error));
```

Async/Await Style

```
async function getData() {
try {
const data = await fetchData();
console.log(data);
}
catch(error) {
console.log(error);
}
}
```

Multiple Await

```
async function loadData() {

const users =
await fetchUsers();

const posts =
await fetchPosts();

const comments =
await fetchComments();

}
```

Parallel Execution

```
async function loadData() {

    const [users, posts] =
        await Promise.all([
            fetchUsers(),
            fetchPosts()
        ]);

}
```

Callback vs Promise vs Async/Await

Feature	Callback	Promise	Async/Await
Readability	Poor	Good	Excellent
Error Handling	Difficult	Easy	Very Easy
Chaining	Difficult	Easy	Easy
Callback Hell	Yes	No	No
Modern Usage	Rare	Common	Most Common

Task

Create a JavaScript program that:

1. Displays **"Download Started..."**
2. After 3 seconds displays **"Downloading..."**
3. After another 2 seconds displays **"Download Completed!"**
4. Use **callback functions** to control the sequence of execution.

Expected Output

Download Started...
Downloading...
Download Completed!